UPRM-Edge

A Minimal Resource-Aware DSL for Edge Deployment — Track Selection Guide

1. Purpose of This Document

This semester, your team will design and implement a small piece of UPRM-Edge: a minimal Domain-Specific Language (DSL) purpose-built for programming resource-constrained edge devices (microcontrollers, TinyML sensor nodes, etc.).

UPRM-Edge shares one common core language across all teams (variables, arithmetic, conditionals, loops, functions — the part everyone implements and that gets autograded). On top of that shared core, each team picks ONE track and adds a resource-aware language feature that does not exist in mainstream languages. This document shows what each track's syntax could look like, so your team can decide which one to pursue.

Read all four tracks before deciding. You are not just picking a project — you are picking which real-world edge computing problem your team wants to attack with a language-design solution.

2. Why "UPRM-Edge"?

The name reflects three things:

- UPRM — this language is designed, specified, and implemented by CIIC 4030 students at the University of Puerto Rico, Mayagüez.
- Edge — its entire reason for existing is programming for edge/embedded/TinyML deployment, not general-purpose computing.
- It is intentionally minimal — think of it as a teaching language, not a production one. The goal is to demonstrate that resource constraints CAN be expressed as first-class language features, not to replace any language.

File extension for all sample programs in this document and in your submissions: .uedge

3. The Common Core (All Tracks Implement This)

Every team's interpreter must support this shared core, regardless of track. This is the part of the language that is identical for everyone and is what makes autograded testing possible.

3.1 Core Language Features

- Variables and assignment: let x = 5;
- Basic types: int, float, bool, string
- Arithmetic and comparison operators: + - * / % == != < > <= >=
- Conditionals: if / else
- Loops: while, for
- Functions: func name(params) { ... } with return

- Output: print(expression);
- Comments: // single-line comments

3.2 Core Syntax Example

```
// core UPRM-Edge program — no track-specific features
func average(a, b) {
    return (a + b) / 2;
}

let readings = 0;
let total = 0;

for (let i = 0; i < 5; i = i + 1) {
    let sample = readSensor();
    total = total + sample;
    readings = readings + 1;
}

print(average(total, readings));
```

Everything below this point is the SAME base grammar with one track-specific extension layered on top. You are not designing a new language from scratch — you are extending this shared core.

4. The Four Tracks

Choose one. Each track adds a different resource-aware feature to the common core above.

Track A — Memory-Aware

Adds a compile-time/runtime memory budget annotation to functions or blocks. The interpreter must estimate memory usage of everything declared inside the annotated scope and flag a violation if the estimate exceeds the stated budget.

Syntax

```
@memory(40kb)
func readSensorBuffer() {
    let buffer = array(int, 1000);   // interpreter estimates size
    let temp = array(float, 200);
    // ... if total estimated allocation > 40kb -> compile-time error
}
```

What your interpreter must do

- Parse the @memory(N unit) annotation attached to a function.
- Track declared array/variable sizes inside that function's scope.
- Sum estimated memory usage and compare against the stated budget.
- Report a clear error (with location) when the budget is exceeded — this is the core "resource-aware" behavior being demonstrated.

Why this matters

Running out of RAM is one of the most common real-world failure modes on microcontrollers, and today nothing in the language itself catches it before deployment — you find out when the device crashes in the field.

Track B — Power-Aware

Adds a power/energy budget annotation. The interpreter uses a simple cost model (you define reasonable relative costs per operation type) and accumulates estimated power/energy draw as the annotated code runs or is statically analyzed.

Syntax

```
@power(2mA)
func transmitReading(value) {
    radioSend(value);   // e.g., costed at 1.5mA in your cost model
    blinkLED();         // e.g., costed at 0.3mA
    // if total estimated draw > 2mA -> flag violation
}
```

What your interpreter must do

- Parse the @power(N unit) annotation.
- Units can be A, J, or Wh
- Maintain a cost table mapping operations/built-in calls (radio, sensor reads, LED, etc.) to an estimated power cost — your team defines these values and documents your reasoning.
- Accumulate cost while executing (or statically walking) the annotated function.
- Flag a violation when the accumulated cost exceeds the declared budget.

Why this matters

Battery-powered field deployments (like environmental sensors) live or die by power budget. Today power budgeting is done by hand in spreadsheets, disconnected from the code that actually determines power draw.

Track C — Precision-Aware

Adds mixed-precision numeric types alongside the normal int/float, and makes the interpreter actually simulate the behavior of reduced precision (overflow, truncation) rather than silently treating everything as full precision.

Syntax

```
func quantizeExample() {
    int8 small = 120;
    small = small + 20;      // exceeds int8 range (-128..127)
    print(small);            // must reflect real int8 overflow behavior

    float16 approx = 3.14159; // reduced precision vs float32
    print(approx);            // must reflect real float16 rounding
```

```
}
```

What your interpreter must do

- Support at least two reduced-precision types (e.g., int8 and float16) in addition to the core int/float.
- Correctly simulate overflow/wraparound or truncation/rounding behavior specific to each reduced type.
- Support explicit casts between precisions, e.g., int8(x) or float16(x).

Why this matters

Quantization (running ML models at int8 instead of float32) is the single biggest technique for fitting models onto microcontrollers. Right now this is applied by external tools after training — no mainstream language lets you express precision as a type you reason about directly.

Track D — Portability-Aware

Adds a @target block that lets one source file describe behavior for two different (simplified/simulated) hardware targets, and the interpreter must compile/execute the same program correctly against both target definitions.

Syntax

```
@target(esp32)
func blink() {
    setPin(2, HIGH);   // ESP32 uses GPIO pin 2 for onboard LED
}

@target(nrf52)
func blink() {
    setPin(17, HIGH);  // nRF52 uses GPIO pin 17 for onboard LED
}

// same call site, resolved differently depending on build target:
blink();
```

What your interpreter must do

- Parse multiple @target(name) implementations of the same function name.
- Accept a target flag (e.g., a command-line argument) selecting which target definitions to use.
- Run the SAME core program twice — once per target — and show both executions produce the correct target-specific behavior from one shared source file.

Why this matters

Moving a project between microcontroller families today usually means a partial rewrite. A language that cleanly separates "what the program does" from "how a specific board does it" attacks a real, everyday portability problem.

5. Track Comparison

Track	What It Enforces	Why It Matters in Real Edge Deployments	Good Fit For
Memory-Aware	Static/dynamic memory-budget checking	RAM/flash overruns are the #1 cause of MCU crashes	Teams comfortable with basic data structures & stack/heap concepts
Power-Aware	Power/energy budget checking per function	Battery life is often the hard constraint in field-deployed sensors	Teams interested in energy modeling / systems thinking
Precision-Aware	Mixed-precision numeric types (int8/float16/float32)	Quantization is central to how ML models run on MCUs	Teams interested in ML/numerical computing
Portability-Aware	Single source, multiple hardware targets	Rewriting code per board family is a major real-world pain point	Teams comfortable with compiler back-end concepts

6. How Each Track Is Graded

Regardless of track, grading follows the same three-part structure across the semester's subprojects:

- Subproject 1 (Lexer/Parser + Oral Report): common core grammar + your track's syntax extension — autograded against hidden test programs.
- Subproject 2 (Interpreter + Oral Report): common core execution + your track's enforcement behavior (memory check, power check, precision simulation, or dual-target compilation) — autograded against hidden test programs, including test cases specifically designed to trigger your track's resource violation.
- Subproject 3 (Hardware Deployment + Oral Report): your team deploys/tests on real hardware, compares your DSL's predictions against real measured behavior, and presents findings live — graded as Oral Report + Project components.

7. What To Submit for Track Selection

By the assigned deadline, each team submits:

- Your chosen track (A, B, C, or D).
- Team member names and roles (at least 3, at most 4)
- A short paragraph (3-5 sentences) on why this track interests your team and any relevant background (e.g., "we've worked with battery-powered sensors before").

If you are unsure, come to office hours — talking through your team's strengths against the four tracks above usually makes the choice obvious quickly.